PYTHON MODÜL 9 · PROJE DOKÜMANI

Cosmic Data

Pydantic modelleri ve veri doğrulama: nasıl çalışır, ne kazandırır, ne öğrendik

Bu proje, uzay temalı üç alıştırma üzerinden Pydantic v2 ile veri doğrulamayı öğretir. Her alıştırma bir öncekinin üzerine tek bir kavram ekler: önce alan kısıtları, sonra alanlar arası iş kuralları, en son iç içe modeller.

ALIŞTIRMA 0

Space Station Data

`BaseModel` ve `Field` ile temel model ve kısıtlar.

ALIŞTIRMA 1

Alien Contact Logs

`Enum` ve `@model_validator` ile iş kuralları.

ALIŞTIRMA 2

Space Crew Management

İç içe modeller, liste kısıtları ve ekip güvenlik kuralları.

1 Proje bir bakışta

Senaryo: Cosmic Data Observatory'de çalışan bir veri mühendisi olarak uzay istasyonlarından, uzaylı temas raporlarından ve görev ekiplerinden gelen verinin sisteme girmeden önce doğru olduğundan emin olmamız gerekiyor. Hatalı veri sessizce kabul edilmemeli, anlaşılır bir mesajla reddedilmeli.

Klasör	Teslim dosyası	Tanımlanan sınıflar	Öğrettiği kavram
`ex0/`	`space_station.py`	`SpaceStation`	Model tanımı, alan kısıtları, varsayılan değer, otomatik tip dönüşümü
`ex1/`	`alien_contact.py`	`ContactType`, `AlienContact`	Enum alanı ve birden fazla alana bakan özel doğrulama
`ex2/`	`space_crew.py`	`Rank`, `CrewMember`, `SpaceMission`	Model içinde model listesi ve liste üzerinden kurallar

Genel kurallar ve projede nasıl karşılandıkları

Konunun istediği	Projedeki karşılığı
Python 3.10 veya üstü	3.10 ile uyumlu sözdizimi kullanıldı; 3.10.22 ve 3.12 üzerinde çalıştırıldı.
flake8 standardı	Üç dosyada sıfır uyarı. En uzun satır 79 karakter.
Eksiksiz tip açıklamaları, mypy	Her fonksiyonun parametre ve dönüş tipi var. Hem `mypy` hem `mypy --strict` hatasız.
Pydantic 2.x, `@validator` yok	Özel kurallar yalnızca `@model_validator(mode="after")` ile yazıldı.
Hata yönetimi veriyi korumalı	Her model oluşturma işlemi `try / except ValidationError` içinde; program çökmez, mesaj basar.
Authorized: None	Pydantic dışında yalnızca standart kütüphane: `datetime`, `enum`, `typing`.
Her alıştırmada `main()`	Geçerli bir örnek gösterir, sonra bilerek hatalı bir örnek dener ve hata mesajını yazdırır.

2 Kurulum ve çalıştırma

Konu sanal ortam ve `pip` kullanılmasını şart koşuyor. Depo kökünde:

```
python3 -m venv venv
source venv/bin/activate              (Windows: venv\Scripts\activate)
pip install "pydantic>=2,<3" flake8 mypy

python3 ex0/space_station.py
python3 ex1/alien_contact.py
python3 ex2/space_crew.py

flake8 ex0 ex1 ex2
mypy ex0 ex1 ex2
```

> **Dikkat:** `flake8` ve `mypy` komutlarına klasör adlarını verin. Yalnızca `flake8 .` yazarsanız `venv/` içindeki binlerce kütüphane dosyası da taranır.

3 Pydantic doğrulaması nasıl çalışır

Üç alıştırmanın hepsi aynı boru hattını kullanır. Bir model sınıfını çağırdığımızda Pydantic veriyi şu sırayla işler:

1 · GİRDİ
Ham veri
Sözlük veya anahtar kelime argümanları. Tipler henüz güvenilir değil.

→

2 · ALANLAR
Tip + Field kısıtları
Her alan tipine çevrilir, sonra uzunluk ve aralık kontrol edilir.

→

3 · MODEL
model_validator
Alanlar geçerliyse çalışır. Alanlar arası iş kurallarına bakar.

→

4 · SONUÇ
Geçerli nesne
Tüm alanları doğru tipte olan, güvenle kullanılabilir nesne.

Hata dalı: 2. veya 3. adımda bir kural bozulursa nesne hiç oluşmaz, `ValidationError` fırlatılır. `main()` bunu `except` ile yakalar ve `error.errors()` listesindeki `msg` alanlarını yazdırır.

Kavram	Ne işe yarar	Projede nerede
`BaseModel`	Sınıfı doğrulanan bir veri modeline çevirir. Tip açıklamaları kural olarak okunur.	Dört model sınıfının hepsi
`Field(...)`	Alana kısıt ve varsayılan ekler. İlk argümandaki `...` "zorunlu alan" demektir.	`min_length`, `max_length`, `ge`, `le`, `default`
Tip dönüşümü	Uyumlu girdiyi hedef tipe çevirir: `"2024-01-15T10:30:00"` metni `datetime` olur.	ex0 ve ex1 tarih alanları
`Enum`	Alanı sabit bir seçenek kümesine kısıtlar. Küme dışındaki değer reddedilir.	`ContactType`, `Rank`
`@model_validator`	Tek alana sığmayan, birden çok alana birlikte bakan kuralları yazdığımız yer.	`AlienContact`, `SpaceMission`
İç içe model	Bir alanın tipi başka bir model olabilir. Pydantic içteki modeli de doğrular.	`crew: list[CrewMember]`
`ValidationError`	Tüm hataları konumu ve mesajıyla taşıyan istisna. `ValueError`'ın alt sınıfıdır.	Her `main()` içindeki `except`

4 Alıştırma 0: Space Station Data

Amaç: `BaseModel` ve `Field` ile ilk modeli kurmak. Burada hiç özel kod yazmadan, yalnızca bildirimle doğrulama yapıyoruz.

```
class SpaceStation(BaseModel):
    """A station report. Field() sets the length and range limits."""

    station_id: str = Field(..., min_length=3, max_length=10)
    name: str = Field(..., min_length=1, max_length=50)
    crew_size: int = Field(..., ge=1, le=20)
    power_level: float = Field(..., ge=0.0, le=100.0)
    oxygen_level: float = Field(..., ge=0.0, le=100.0)
    last_maintenance: datetime
    is_operational: bool = Field(default=True)
    notes: Optional[str] = Field(default=None, max_length=200)
```

Alan	Konudaki tanım	Koddaki karşılığı
`station_id`	Metin, 3-10 karakter	`min_length=3, max_length=10`
`name`	Metin, 1-50 karakter	`min_length=1, max_length=50`
`crew_size`	Tam sayı, 1-20 kişi	`ge=1, le=20` (büyük eşit, küçük eşit)
`power_level`, `oxygen_level`	Ondalık, yüzde 0.0-100.0	`ge=0.0, le=100.0`
`last_maintenance`	Tarih-saat	Tip `datetime`; kısıt gerekmediği için `Field` yok
`is_operational`	Mantıksal, varsayılan True	`Field(default=True)`
`notes`	İsteğe bağlı metin, en çok 200	`Optional[str]`, `default=None, max_length=200`

main() ne yapar

1. Geçerli veriyi bir sözlükte tutar. Tarih bilerek metin olarak verilir.
2. `SpaceStation(**station_data)` ile nesneyi oluşturur ve bilgileri yazdırır.
3. Aynı sözlüğün `crew_size` değeri 25 olan bir kopyasını dener.
4. Pydantic `le=20` kısıtı yüzünden `ValidationError` fırlatır; mesaj yazdırılır.

Program çıktısı

```
Space Station Data Validation
========================================
Valid station created:
ID: ISS001
Name: International Space Station
Crew: 6 people
Power: 85.5%
Oxygen: 92.3%
Status: Operational

========================================
Expected validation error:
Input should be less than or equal to 20
```

> **Konudaki "Think About" sorusu:** Pydantic varsayılan olarak esnek modda çalışır ve anlamı net olan girdileri hedef tipe çevirir. ISO biçimindeki `"2024-01-15T10:30:00"` metni gerçek bir `datetime` nesnesine, `"6"` metni `6` tam sayısına, `92` tam sayısı `92.0` ondalığına dönüşür. Çevrilemeyen girdi (örneğin `"yesterday"`) sessizce geçmez, hata üretir.

5 Alıştırma 1: Alien Contact Logs

Amaç: Tek bir alana bakarak karar verilemeyen kuralları `@model_validator` ile yazmak.

Örneğin "telepatik temasta en az 3 tanık" kuralı hem `contact_type` hem `witness_count` alanına bakar; bunu `Field` ile ifade edemeyiz.

```
class ContactType(str, Enum):
    """Allowed contact types."""

    RADIO = "radio"
    VISUAL = "visual"
    PHYSICAL = "physical"
    TELEPATHIC = "telepathic"
```

```
    @model_validator(mode="after")
    def validate_business_rules(self) -> "AlienContact":
        """Business rules, checked once the fields are valid."""
        is_physical = self.contact_type == ContactType.PHYSICAL
        is_telepathic = self.contact_type == ContactType.TELEPATHIC
        if not self.contact_id.startswith("AC"):
            raise ValueError('Contact ID must start with "AC" (Alien Contact)')
        if is_physical and not self.is_verified:
            raise ValueError("Physical contact reports must be verified")
        if is_telepathic and self.witness_count < 3:
            raise ValueError(
                "Telepathic contact requires at least 3 witnesses"
            )
        if self.signal_strength > 7.0 and not self.message_received:
            raise ValueError(
                "Strong signals (> 7.0) should include received messages"
            )
        return self
```

#	İş kuralı	Reddedilen durum
1	Kimlik "AC" ile başlamalı	`contact_id` başka bir önekle başlıyorsa
2	Fiziksel temas doğrulanmış olmalı	Tür `physical` ve `is_verified` False ise
3	Telepatik temas en az 3 tanık ister	Tür `telepathic` ve `witness_count` 3'ten küçükse
4	Güçlü sinyal mesaj içermeli	`signal_strength` 7.0'dan büyük ve mesaj yok ya da boş ise

Dikkat edilen ayrıntılar

- **mode="after":** Fonksiyon, alanlar doğrulanıp nesne oluştuktan sonra çalışır. Bu yüzden `self.witness_count` kesin olarak tam sayıdır, `self.contact_type` kesin olarak enum üyesidir.
- **return self:** Pydantic, doğrulayıcının döndürdüğü değeri sonuç kabul eder. Unutulursa Pydantic uyarı verir ve `model_validate()` nesne yerine `None` döndürür.
- **Sınır değeri:** Kural "7.0'dan büyük" dediği için tam 7.0 mesajsız kabul edilir, 7.1 edilmez.

Program çıktısı

```
Alien Contact Log Validation
====================================
Valid contact report:
ID: AC_2024_001
Type: radio
Location: Area 51, Nevada
Signal: 8.5/10
Duration: 45 minutes
Witnesses: 5
Message: 'Greetings from Zeta Reticuli'

====================================
Expected validation error:
Telepathic contact requires at least 3 witnesses
```

> **Önemli davranış:** Alan hataları toplu raporlanır; dört alan bozuksa dördü birden listelenir. Model doğrulayıcısı ise yalnızca tüm alanlar geçerliyse çalışır ve ilk `raise` satırında durur. Yani aynı anda iki iş kuralı bozuksa yalnızca ilki raporlanır.

> **Mesaj öneki:** Pydantic, `ValueError` mesajlarını `"Value error, "` önekiyle sarar. Beklenen çıktıda bu önek olmadığı için `display_errors` içinde `removeprefix` ile temizlenir.

6 Alıştırma 2: Space Crew Management

Amaç: Modelleri iç içe kullanmak. Bir görev (`SpaceMission`) kendi içinde ekip üyelerinin (`CrewMember`) listesini taşır. Pydantic önce her üyeyi kendi kurallarıyla doğrular, sonra görev seviyesindeki kurallara geçer.

```
class CrewMember(BaseModel):
    """One crew member, validated on its own before the mission."""

    member_id: str = Field(..., min_length=3, max_length=10)
    name: str = Field(..., min_length=2, max_length=50)
    rank: Rank
    age: int = Field(..., ge=18, le=80)
    specialization: str = Field(..., min_length=3, max_length=30)
    years_experience: int = Field(..., ge=0, le=50)
    is_active: bool = Field(default=True)


class SpaceMission(BaseModel):
    """A mission and its crew. Each crew item is a CrewMember model."""

    mission_id: str = Field(..., min_length=5, max_length=15)
    mission_name: str = Field(..., min_length=3, max_length=100)
    destination: str = Field(..., min_length=3, max_length=50)
    launch_date: datetime
    duration_days: int = Field(..., ge=1, le=3650)
    crew: list[CrewMember] = Field(..., min_length=1, max_length=12)
    mission_status: str = Field(default="planned")
    budget_millions: float = Field(..., ge=1.0, le=10000.0)
```

`crew` alanındaki `min_length=1, max_length=12` metin uzunluğu değil, listedeki eleman sayısıdır: boş ekip ve 13 kişilik ekip reddedilir.

```
    @model_validator(mode="after")
    def validate_safety_requirements(self) -> "SpaceMission":
        """Safety rules that need the whole crew list."""
        leading_ranks = (Rank.COMMANDER, Rank.CAPTAIN)
        leaders = [m for m in self.crew if m.rank in leading_ranks]
        experienced = [m for m in self.crew if m.years_experience >= 5]
        if not self.mission_id.startswith("M"):
            raise ValueError('Mission ID must start with "M"')
        if not leaders:
            raise ValueError(
                "Mission must have at least one Commander or Captain"
            )
        if self.duration_days > 365 and len(experienced) * 2 < len(self.crew):
            raise ValueError(
                "Long missions (> 365 days) need 50% experienced crew "
                "(5+ years)"
            )
        if not all(member.is_active for member in self.crew):
            raise ValueError("All crew members must be active")
        return self
```

#	Güvenlik kuralı	Kodda nasıl denetleniyor
1	Görev kimliği "M" ile başlamalı	`mission_id.startswith("M")`
2	En az bir Commander veya Captain	Rütbesi bu ikisinden biri olanların listesi boş olmamalı
3	365 günden uzun görevde ekibin %50'si deneyimli (5+ yıl)	Deneyimli sayısının iki katı, ekip sayısından az olmamalı
4	Tüm ekip aktif olmalı	`all(member.is_active ...)`

%50 kuralı neden çarpma ile

`len(experienced) * 2 < len(self.crew)` ifadesi "deneyimli oranı yarıdan az" demektir. Bölme yerine çarpma kullanıldığı için ondalık yuvarlama sorunu olmaz.

Örnek görev 900 gün sürüyor ve 3 kişilik. Deneyim yılları 20, 9 ve 4. Deneyimli sayısı 2; 2 x 2 = 4, bu 3'ten küçük olmadığı için görev geçerli. Yalnızca 1 deneyimli olsaydı 2 < 3 olur ve görev reddedilirdi.

main() ne yapar

Önce komutanı olan geçerli Mars görevini kurar. Sonra yalnızca bir teğmen ve bir subaydan oluşan ekiple ikinci bir görev dener; 2. kural bozulduğu için hata yazdırılır.

Program çıktısı

```
Space Mission Crew Validation
========================================
Valid mission created:
Mission: Mars Colony Establishment
ID: M2024_MARS
Destination: Mars
Duration: 900 days
Budget: $2500.0M
Crew size: 3
Crew members:
- Sarah Connor (commander) - Mission Command
- John Smith (lieutenant) - Navigation
- Alice Johnson (officer) - Engineering

========================================
Expected validation error:
Mission must have at least one Commander or Captain
```

Konudaki "Think About" sorusu: İç içe doğrulama içten dışa çalışır. Ekip listesindeki her eleman, ister hazır `CrewMember` nesnesi ister ham sözlük olsun, önce `CrewMember` kurallarından geçer. Bir üye geçersizse görev hiç oluşturulmaz ve hata, sorunun tam yerini gösterir: 17 yaşında bir ikinci üye için konum `crew.1.age`, mesaj `Input should be greater than or equal to 18` olur. Bu durumda görev seviyesindeki doğrulayıcı hiç çalışmaz.

7 Test ve doğrulama sonuçları

Kod teslimden önce aşağıdaki kontrollerden geçirildi. Ortam: Python 3.10.22, pydantic 2.14.0, flake8 7.4.1, mypy 2.4.0. Çıktı, flake8 ve mypy kontrolleri ayrıca Python 3.12 üzerinde de tekrarlandı ve aynı sonucu verdi.

Kontrol	Sonuç	Ayrıntı
Çıktının konudaki örnekle karşılaştırılması	3 / 3 aynı	Üç programın çıktısı, beklenen örnekle karakter karakter eşleşiyor. Ayraç uzunlukları 40, 38 ve 41.
`flake8 ex0 ex1 ex2`	0 uyarı	Varsayılan ayarlarla.
`mypy ex0 ex1 ex2`	Hatasız	3 kaynak dosyada sorun bulunmadı.
`mypy --strict`	Hatasız	Konu istemiyor; ek güvence olarak çalıştırıldı.
Kural ve sınır değeri denemeleri	56 / 56	Her alanın alt ve üst sınırı ile her iş kuralının iki yanı denendi.
Kod içi belgeleme	Docstring	Modül, sınıf, doğrulayıcı ve `main()` için kısa İngilizce docstring'ler var.

Sınır değeri denemelerinden örnekler

Alş.	Denenen girdi	Karar	Pydantic'in mesajı
ex0	`crew_size` 20 / 21	kabul / ret	Input should be less than or equal to 20
ex0	`station_id` 2 karakter	ret	String should have at least 3 characters
ex0	`notes` 200 / 201 karakter	kabul / ret	String should have at most 200 characters
ex1	`contact_type` "smoke"	ret	Input should be 'radio', 'visual', 'physical' or 'telepathic'
ex1	Telepatik, 2 / 3 tanık	ret / kabul	Telepathic contact requires at least 3 witnesses
ex1	Sinyal 7.0 / 7.1, mesaj yok	kabul / ret	Strong signals (> 7.0) should include received messages
ex1	Fiziksel, doğrulanmamış	ret	Physical contact reports must be verified
ex2	Ekip 12 / 13 kişi	kabul / ret	List should have at most 12 items after validation, not 13
ex2	900 gün, 2 kişiden 1'i / 3 kişiden 1'i deneyimli	kabul / ret	Long missions (> 365 days) need 50% experienced crew (5+ years)
ex2	Tam 365 gün, 3 kişiden 1'i deneyimli	kabul	Kural yalnızca 365 günden uzun görevlerde devreye girer.
ex2	Ekipte pasif üye	ret	All crew members must be active
ex2	İç içe üyede yaş 17	ret	`crew.1.age`: Input should be greater than or equal to 18

8 Kazanımlar

Bu projeyi tamamlayan biri şunları yapabilir hale gelir:

- **Veri modeli tasarlamak:** Bir gereksinim listesini ("3-10 karakter", "1-20 kişi") doğrudan tip açıklaması ve `Field` kısıtlarına çevirmek.
- **Bildirimsel doğrulama:** `if` zincirleri yazmak yerine kuralı alanın tanımına koymak ve denetimi kütüphaneye bırakmak.
- **İş kuralı yazmak:** Alanlar arası koşulları `@model_validator(mode="after")` içinde, okunabilir hata mesajlarıyla ifade etmek.
- **Seçenek kümelerini güvenceye almak:** Serbest metin yerine `Enum` kullanarak yazım hatalarını girişte yakalamak.
- **Bileşik veri kurmak:** Modelleri iç içe kullanmak ve hatanın konumunu (`crew.1.age`) okuyabilmek.
- **Hatayı yönetmek:** `ValidationError`'ı yakalamak, `errors()` listesinden mesajı çıkarmak ve programı çökertmeden devam etmek.
- **Kod kalitesi araçları:** Sanal ortam kurmak, `pip` ile bağımlılık yüklemek, `flake8` ve `mypy` çıktısını temiz tutmak.

9 Ne öğrendik

- **Doğrulama sınırda yapılır.** Veri sisteme girerken bir kez sıkı denetlenirse, içerideki kodun tekrar tekrar kontrol yazmasına gerek kalmaz. Geçerli bir `SpaceMission` nesnesi elimizdeyse ekibinde en az bir lider olduğundan eminiz.
- **Tip açıklamaları çalışma zamanında da işe yarar.** Normal Python'da `crew_size: int` yalnızca bir ipucudur. Pydantic'te aynı satır gerçek bir kuraldır.
- **Kuralın yeri önemlidir.** Tek alanla ilgili kural `Field` içine, birden fazla alanla ilgili kural model doğrulayıcısına yazılır. Yanlış yere konan kural ya çalışmaz ya da okunmaz hale gelir.
- **Dönüşüm ile doğrulama aynı şey değildir.** Pydantic önce çevirmeyi dener, sonra denetler. Bu, JSON ve CSV gibi her şeyin metin geldiği kaynaklarla çalışmayı kolaylaştırır.
- **İyi hata mesajı özelliğin parçasıdır.** "Hatalı veri" demek yerine hangi alanın, neden reddedildiğini söyleyen mesaj, sorunu kaynağında düzeltmeyi sağlar.
- **Opsiyonel olmak ile None alabilmek farklıdır.** `Optional[str]` yalnızca `None` değerine izin verir. Alanı gerçekten atlanabilir yapan şey `default=None` vermektir; varsayılan verilmezse alan hâlâ zorunludur.

Pydantic v1 ve v2 farkı

Konu	v1 (kullanılmadı)	v2 (bu projede)
Çok alanlı özel doğrulama	`@validator`, `@root_validator`	`@model_validator(mode="after")`
Liste eleman sayısı kısıtı	`min_items`, `max_items`	`min_length`, `max_length`
Doğrulayıcının erişimi	`values` sözlüğü	Doğrudan `self`, tipli alanlarla

10 Değerlendirmede sorulabilecekler

Field(...) içindeki üç nokta ne anlama geliyor?

Python'ın `Ellipsis` nesnesidir. Pydantic'e "bu alanın varsayılanı yok, mutlaka verilmeli" der. Varsayılanı olan alanlarda yerine `default=True` gibi bir değer yazılır.

mode="after" ile mode="before" arasındaki fark nedir?

`before` ham girdi üzerinde, tip dönüşümünden önce çalışır; veriyi düzeltmek için kullanılır. `after` alanlar doğrulandıktan sonra çalışır ve `self` üzerinden doğru tipteki değerlere erişir. İş kuralları doğrulanmış veriye baktığı için `after` seçildi.

Dönüş tipi neden tırnak içinde "AlienContact" yazılmış?

Fonksiyon tanımlanırken sınıfın kendisi henüz tamamlanmamıştır, bu yüzden ad metin olarak verilir. `typing.Self` Python 3.11 ile geldi; proje 3.10'u desteklemek zorunda olduğu için bu yol seçildi.

Enum sınıfları neden hem str hem Enum'dan türüyor?

Üyeler böylece aynı zamanda metindir: `ContactType.RADIO == "radio"` doğrudur ve JSON'a yazılırken ek işlem gerekmez. Pydantic girdideki `"radio"` metnini otomatik olarak enum üyesine çevirir.

Geçerli örneklerde veri neden sözlük olarak hazırlanıp ** ile açılıyor?

ex0 ve ex1'de tarih bilerek metin verilir ki tip dönüşümü görülsün. Metni doğrudan `datetime` parametresine yazmak mypy hatası üretir; `dict[str, Any]` üzerinden vermek hem gerçek dünyadaki JSON girdisini taklit eder hem de mypy'ı temiz tutar. ex2'de ise nesneler tipli argümanlarla doğrudan kurulur.

Geçersiz örnekte neden yalnızca tek hata görünüyor?

Örnekte tek bir kural bozuldu. Birden çok alan bozuk olsaydı Pydantic hepsini listelerdi; `display_errors` fonksiyonu bu yüzden bir döngüyle tüm hataları yazdırır.

11 Teslim notları

- **Yalnızca üç dosya teslim edilir:** `ex0/space_station.py`, `ex1/alien_contact.py`, `ex2/space_crew.py`. Konu "yalnızca istenen dosyalar" dediği için bu doküman, konu PDF'i ve `venv/` klasörü depoya eklenmemeli.
- **Kod içi belgeleme:** Konunun III.2 bölümü öğrenme sürecinin "açık kod yorumları ve anlamlı değişken adlarıyla" belgelenmesini ister. Bu projede yorumlar kısa İngilizce docstring olarak yazıldı: her dosyanın başında, her sınıfta, doğrulayıcılarda ve `main()` fonksiyonunda birer satır. Gerisini anlamlı adlar (`overcrowded_data`, `leaderless_crew`, `validate_safety_requirements`) anlatır.
- **Veri üretme araçları:** Konuda adı geçen `data_generator.py`, `data_exporter.py` ve `generated_data/` proje klasöründe yoktu. Kullanımları zorunlu olmadığı için örnek veriler doğrudan `main()` içinde tanımlandı.